A Lightweight Authentication Protocol Against Modeling Attacks Based on a Novel LFSR-APUF

Yao Wang, *Member, IEEE*, Xue Mei, Zhengtai Chang, Wenbing Fan, Benqing Guo, *Member, IEEE*, Zhi Quan, and Deepak Kumar Jain, *Senior Member, IEEE*

***Abstract*—Simple authentication protocols based on conventional physical unclonable functions (PUFs) are vulnerable to modeling attacks and other security threats. This article proposes an arbiter PUF based on a linear feedback shift register (LFSR-APUF). Different from the previously reported linear feedback shift register (LFSR) for challenge extension, the proposed scheme feeds the external random challenges into the LFSR module to obfuscate the linear mapping relationship between the challenge and response. It can prevent attackers from obtaining valid challenge–response pairs (CRPs), increasing its resistance to modeling attacks significantly. A 64-stage LFSR-APUF has been implemented on a field programmable gate array (FPGA) board. The experimental results reveal that the proposed design can effectively resist various modeling attacks, such as logistic regression (LR), evolutionary strategy (ES), artificial neuro network (ANN), and support vector machine (SVM) with a prediction rate of 51.79% and a slight effect on the randomness, reliability, and uniqueness. Further, a lightweight authentication protocol is established based on the proposed LFSR-APUF. The protocol incorporates a low-overhead, ultralightweight, novel private bit conversion *Cover* function that is uniquely bound to each device in the authentication network. The proposed authentication protocol not only resists spoofing attacks, physical attacks, and modeling attacks effectively but also ensures the security of the entire authentication network by transferring important information in encrypted form from the server to the database even when the attacker completely controls the server.**

***Index Terms*—Authentication protocol, hardware security, modeling attack, physical unclonable function (PUF).**

Manuscript received 15 May 2023; revised 28 August 2023; accepted 2 September 2023. Date of publication 12 September 2023; date of current version 25 December 2023. This work was supported in part by the National High Technology Research and Development Program of China under Grant SQ2020YFF0404465, and in part by the Science and Technology Project of Henan Province under Grant 232102211076. *(Corresponding authors: Yao Wang; Zhi Quan.)*

Yao Wang, Xue Mei, Zhengtai Chang, Wenbing Fan, and Zhi Quan are with the School of Electrical and Information Engineering, Zhengzhou University, Zhengzhou 450001, China (e-mail: ieyaowang@zzu.edu.cn; mx690774079@163.com; changzhengtai@163.com; iewbfan@zzu.edu.cn; iezquan@zzu.edu.cn).

Benqing Guo is with the College of Communication Engineering, Chengdu University of Information Technology, Chengdu 610000, China (e-mail: rficgbq@gmail.com).

Deepak Kumar Jain is with the Key Laboratory of lntelligent Control and Optimization for Industrial Equipment of Ministry of Education, School of Artificial Intelligence, Dalian University of Technology, Dalian 116024, China, and also with the Symbiosis Institute of Technology, Symbiosis International University, Pune 412115, India (e-mail: dkj@ieee.org).

Digital Object Identifier 10.1109/JIOT.2023.3314058

I. Introduction

THE Internet of Things (IoT) connects and integrates the physical world and digital space based on the existing and evolving information and communication technologies, and it is usually considered a worldwide network of interconnected objects uniquely addressable based on standard communication protocols [1]. However, as the scale of IoT continues to expand and the number and types of access devices gradually increase, information security in IoT devices is becoming a major concern for users. This issue can be addressed by authentication protocols based on various encryption algorithms, such as AES and RSA [2]. The secret key, which is critical for the encryption algorithms and should remain strictly confidential, is commonly stored in nonvolatile memory (NVM). However, some IoT devices are highly resource-constrained with low cost, small size, and ultralow-power consumption, making it difficult to implement complex encryption algorithms. Furthermore, the NVM storing the secret key is vulnerable to various physical attacks, including side-channel, semi-invasive, and invasive attacks [3], [4].

Physical unclonable functions (PUFs) are promising hardware security primitives for resource-constrained systems [5]. Specifically, PUF employs the uncontrollable manufacturing deviations at the deep submicron level to produce unique challenge–response pairs (CRPs) as fingerprints for each instance. The ability to control the precise generation time of the secret bitstring and the sensitivity of the PUF entropy source to invasive probing attacks (which would cause changes in the electrical characteristics of the PUF circuits and make the PUF invalid) are additional attributes that make them attractive for authentication in embedded hardware. PUFs can be divided into two categories: 1) strong PUFs (SPUFs) and 2) weak PUFs (WPUFs) [6]. SPUFs, represented by arbiter PUF (APUF), have more CRPs and are usually employed for authentication purposes.

Although SPUF-based protocols provide an extremely effective and lightweight solution for the authentication of IoT devices, they are not completely reliable and are vulnerable to some serious security attacks, such as modeling attacks. Back in 2010, Rührmair et al. [7] used computational algorithms to simulate some PUF behaviors for the CRP data set, and then they used logistic regression (LR) and evolution strategies (ESs) to successfully break standard APUFs and ring oscillator PUFs as well as XOR APUFs, feed-forward

2327-4662 © 2023 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Zhengzhou University. Downloaded on January 02,2024 at 07:24:36 UTC from IEEE Xplore. Restrictions apply.

APUFs, and lightweight Secure PUFs. Especially, LR could break 64-stage APUF with 99.9% prediction accuracy based on over 18 000 CRPs.

To protect SPUF-based authentication protocols from possible modeling attacks, some modified structures have been reported. These structures can be roughly divided into two categories: 1) enhancing the PUF structure's nonlinearity or 2) obfuscating the CRPs. Each delay chain of conventional APUF is the linear cumulative total of its delay element, causing vulnerability to modeling attacks. To break the linearity of the conventional APUF and enhance its nonlinearity, particular nonlinear components are introduced into APUF [8], [9], [10]. However, variations in the PUF's internal elements are generally unsuitable for field programmable gate array (FPGA) implementation. CRPs obfuscation can also improve the MLA resistance ability of PUF. It only introduces specific mapping circuits at the challenge input or response output ports of the PUF core to obfuscate the real mapping relationship between the challenges and responses, which prevents the attackers from obtaining effective CRPs [11], [12], [13], [14]. However, obfuscation modules, usually have significant hardware overhead. Besides, the obfuscation technique essentially employs a fixed mapping that cannot effectively resist advanced MLAs [15], [16].

In addition to the SPUF-based authentication protocol's vulnerability to modeling attacks, most of the above protocols [11], [12], [13], [14] assume that the server is secure during the authentication phase, i.e., the server cannot be compromised by any attacker. This is because the server stores CRPs regarding SPUF or the software model of SPUF. Once the server is broken in the authentication network, the attacker can obtain information related to the SPUF and other encryption functions, which can easily destroy the entire authentication protocol.

In this study, an APUF based on an linear feedback shift register (LFSR), called LFSR-APUF, is proposed to increase the resistance to modeling attacks. Furthermore, a new lightweight authentication protocol against modeling attacks is established based on the LFSR-APUF. The significant contributions of the work are summarized as follows.

1) An obfuscation scheme is proposed, which can be utilized with existing SPUFs to improve the modeling attack resistance. FPGA experiments indicate that the proposed LFSR-APUF significantly decreases the prediction rate of modeling attacks. The prediction rate of LR, ES, artificial neuro network (ANN), and support vector machine (SVM) can be reduced to nearly 50% by setting relevant parameters.
2) A 64-stage LFSR-APUF is implemented on the Xilinx Artix-7 FPGA board. An unbiased APUF hard macro design methodology is described. This methodology not only ensures the symmetry of the two delay paths of the underlying APUF but also facilitates the modification of obfuscation logic, such as LFSR, without impacting the delay of the underlying APUF. Furthermore, the randomness, uniqueness, reliability, and authentication capacity of the LFSR-APUF circuit are evaluated.

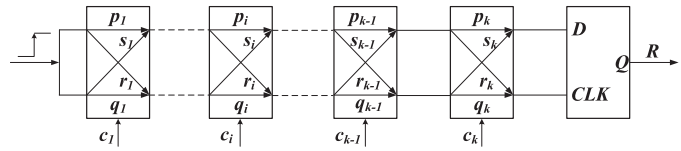

Fig. 1. Structure diagram of k-stage APUF.

3) A new lightweight authentication protocol is proposed, which makes full use of the special characteristic of LFSR-APUF to implement the dynamic obfuscation scheme and further improve the protocol-level modeling attack resistance. The proposed protocol model includes several devices, a server, and a database, any of which can be attacked by attackers. To protect the transmission of CRP information over communication links (i.e., between devices and server as well as between server and database), a new ultralightweight bitstream encryption function *Cover* is established.

The remainder of this article is arranged as follows. Section II introduces the principle and mathematical model of APUF. Section III introduces the proposed LFSR-APUF. The experimental results, including various modeling attacks, randomness, reliability, and uniqueness, are presented in Section IV. Section V presents the proposed lightweight authentication protocol based on LFSR-APUF. The security of this authentication protocol is analyzed in Section VI. Finally, the study is concluded in Section VII.

II. Related Work

As shown in Fig. 1, conventional APUF [5] consists of k-stage cascaded switch blocks, and all of them are entirely symmetric to ensure that the two paths have similar nominal delays. The challenge c_i of APUF acts as the selected signal of the switch block's path to form two different delay chains. If c_i is 0, the upper and lower pulses pass through the switch block in parallel; if c_i is 1, the upper and lower pulses cross through the switch block. The D flip-flop at the terminal of two delay chains generates a response according to the arrival time difference between the top and bottom pulses. If the top pulse arrives at the D port of D flip-flop earlier, R is equal to 1; otherwise, R is 0.

Although a conventional k-stage APUF can generate 2^k CRPs, different challenges have the same k-stage switch blocks. Therefore, the conventional APUF may be considered a linear model, causing a strong correlation between different CRPs. As shown in Fig. 1, the delay of the four signal paths of the ith switch block are denoted by p_i, q_i, s_i, and r_i, and the final R can be formulated as [17]

$$R = \text{sign}(\Delta) = \text{sign}\left(\overrightarrow{\Omega}^T \overrightarrow{\Phi}\right) \tag{1}$$

where sign(x) is the sign function,when $x < 0$, sign(x)=0, otherwise sign(x)=1

$$\overrightarrow{\Omega} = \left(\omega^1, \omega^2, \ldots, \omega^{n+1}\right) \tag{2}$$

Authorized licensed use limited to: Zhengzhou University. Downloaded on January 02,2024 at 07:24:36 UTC from IEEE Xplore. Restrictions apply.

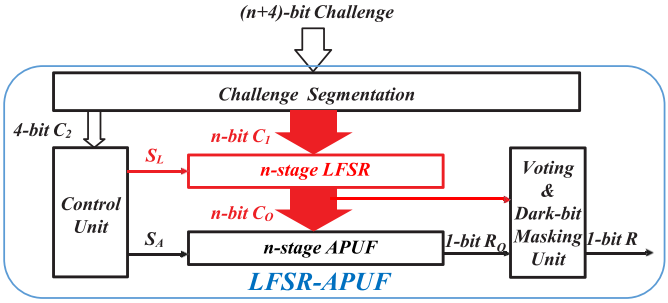

Fig. 2. Schematic of proposed the LFSR-APUF.

where

$$\omega^1 = \frac{\delta_1^0 - \delta_1^1}{2}, w^i = \frac{\delta_{i-1}^0 + \delta_{i-1}^1 + \delta_i^0 - \delta_i^1}{2} \tag{3}$$

$$\delta_i^0 = p_i - q_i, \delta_i^1 = s_i - r_i \tag{4}$$

$$\overrightarrow{\Phi} = \left(\Phi^1, \Phi^2, \ldots, \Phi^k, 1\right)^T \tag{5}$$

$$\Phi^l = \prod_{i=1}^{k}(1 - 2c_i). \tag{6}$$

Ω is a vector representing the delay of multiple switch blocks. It provides a compact model for the process variation of PUF. ω represents the delay of a certain switch block. Φ represents the result of a reversible challenge transformation. If $\Delta = \overrightarrow{\Omega}^T \overrightarrow{\Phi} = 0$, it can be abstracted into a mathematical k-dimensional hyperplane. Any challenge falling on one side of the hyperplane generates the response "0;" otherwise, the response is "1." The attacker can fit this hyperplane by learning some known CRPs through machine learning, and this hyperplane can help him/her predict the corresponding response bit for unknown challenges.

III. Proposed LFSR-APUF

The conventional time-invariant obfuscation scheme converts the external challenge into the obfuscated challenge with some static mapping, while the proposed time-variable obfuscation scheme changes the mapping according to the external challenge in real-time.

A. Design of Proposed LFSR-APUF

As shown in Fig. 2, the proposed LFSR-APUF can be divided into the following five parts.

1) *Challenge Segmentation:* This module divides an $(n+4)$-bit challenge data into two parts, n-bit C_1 and 4-bit C_2, which are sent to LFSR and the Control Unit, respectively.
2) *APUF:* A conventional APUF circuit with n-stage symmetrical switch blocks.
3) *LFSR:* The LFSR is used to obfuscate external challenges.
4) *Control Unit:* This unit controls the LFSR according to the external challenge, and controls the APUF to generate a response.

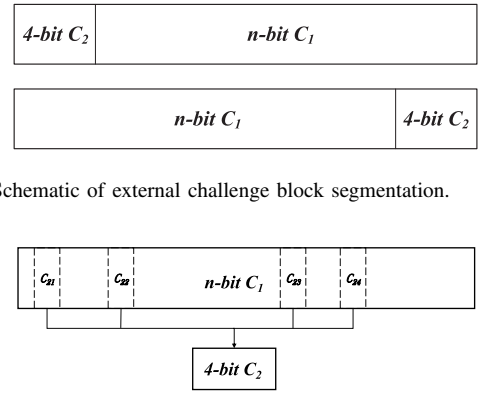

Fig. 3. Schematic of external challenge block segmentation.

Fig. 4. Schematic of external challenge scattered segmentation.

5) *Voting and Dark-Bit Masking Unit:* Voting and dark-bit masking [23] schemes are used on the proposed APUF to improve reliability.

The operating principle of the proposed LFSR-APUF is summarized as follows.

1) The *Challenge Segmentation Unit* divides the $(n+4)$-bit challenge C into n-bit C_1 and 4-bit C_2. C_1 and C_2 are considered the inputs of the n-stage *LFSR Unit* and the *Control Unit*, respectively. Note that C_1 is not all-zeros or all-ones, otherwise the LFSR will be stuck.
2) The n-stage *LFSR Unit* receives the n-bit C_1 generated by the *Challenge Generator* and considers it as the initial value. Then, it waits for the shift signal S_L sent by the *Control Unit*.
3) The *Control Unit* receives the remaining 4-bit C_2 and determines the number of shifts of the LFSR. For example, when C_2 is equal to 0110, the LFSR should be shifted six times, and its final output is employed as the n-bit obfuscated challenge C_O. Finally, when the *Control Unit* sends the shift signal S_L a specific number of times, it sends the pulse signal S_A, namely, the rising edge transition signal, to the n-stage *APUF*.
4) The n-stage APUF receives n-bit obfuscated challenge C_o generated by LFSR and the *Control Unit*'s pulse signal S_A. As shown in Fig. 1, when the pulse signal is transmitted through two delay chains from the start to the final D flip-flop, the *APUF* generates the response R_o. $VotingandDark-bitMasking\ Unit$ screens out unreliable CRPs and outputs reliable response R.

B. Challenge Segmentation Strategy

In the proposed design scheme, $(n+4)$-bit C is categorized into two parts. In the first part, n-bit C_1 is utilized as the LFSR's initial value, while in the second part, 4-bit C_2 determines the shift count of the LFSR and applies it to the Control Unit. As shown in Figs. 3 and 4, these two parts can be roughly divided into the following segmentations.

1) *Block segmentation* refers to setting 4-bit C_2 as a block at the beginning, end, or middle of $(n+4)$-bit C (see Fig. 3 where the C_2 block is located at the starting position of the C).

Authorized licensed use limited to: Zhengzhou University. Downloaded on January 02,2024 at 07:24:36 UTC from IEEE Xplore. Restrictions apply.

2) *Scattered segmentation* refers to scattering 4-bit C_2 within $(n+4)$-bit C. As shown in Fig. 4, C_{21}, C_{22}, C_{23}, and C_{24}, randomly embedded in n-bit C_1.

Different external challenges can be employed to design an obfuscation scheme according to different division strategies for different systems and applications. In this article, the more concise segmentation method, i.e., the block segmentation method is employed. Because the implementation of the block segmentation method consumes less hardware resources.

The value of 4-bit C_2 determines the shift count of the n-stage LFSR. For simplicity, the value of C_2 is chosen to be equal to the *Shift Count* of the LFSR. Thus, there are 16 shift count possibilities. A base integer corresponding to the *Shift Count* is predefined to further enhance the modeling attack resistance of the obfuscation scheme with the LFSR. Now, the final *Shift Count* can be formulated as follows:

$$\text{Shift Count} = C_2 * \text{Base} \tag{7}$$

where C_2 is a 4-bit binary number, *Base* is a predefined integer value, and *Shift Count* is the ultimate number of LFSR shifting controlled by shift signal S_L. If the *Base* is equal to 100, the *Shift Count* should be increased to 100 times the value of C_2 after each external challenge input.

LFSR includes a shift register and a linear feedback function employed as a pseudo-random sequence generator. The primary input of the LFSR is called the seed, while the bits affecting the LFSR's next state are called taps. LFSR can be divided into *Fibonacci* LFSR and *Galois* LFSR according to different feedback methods [24]. Since the hardware implementation of the *Galois* LFSR is generally faster than that of the *Fibonacci* LFSR due to the reduction in the number of logic gates in its feedback loop, the former is chosen in this work.

A time-variant obfuscation scheme is proposed here. Several bits are extracted from the external challenge and applied to the mapping before the external challenge obfuscation. The specific external challenge in real-time is then used to determine the mapping of the obfuscation module. This is equivalent to changing the fixed mapping into a mapping varying with the external challenge and transforming the obfuscation mechanism from "time-invariant" to "time-variant," which significantly reduces the attacker's ability to break it by modeling attack. Moreover, the LFSR-based obfuscation logic can be easily updated by changing the tap sequence of the LFSR to resist the replay attack.

C. Implementation of the Proposed LFSR-APUF on FPGA

Although various original and improved PUF structures have been proposed in the literature, some issues regarding PUF implementation, especially implementation on FPGA, warrant further investigation. The APUF operates by comparing the difference between two delay chains with a similar nominal delay based on the design process but different accurate delays generated by the construction process. However, the placement and routing of APUF are confined by the internal structure of FPGA as a kind of semi-custom integrated circuit, leading to the severe asymmetry of APUF implemented on FPGA.

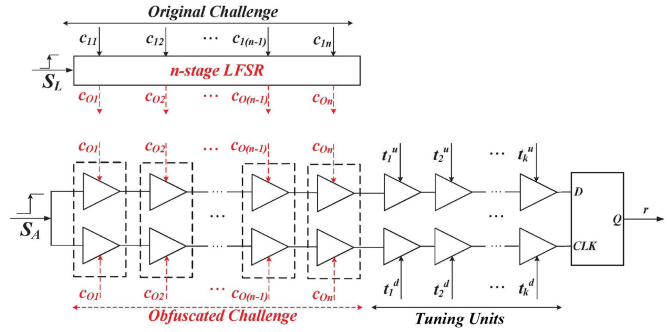

Fig. 5. PDL-based LFSR-PUF structure.

The proposed LFSR-APUF is implemented on Xilinx Artix-7 FPGA. FPGA provides a generic substrate of interconnected blocks that can be (re)programmed several times. Two independent programmable delay lines (PDLs) with low overhead are utilized to introduce a nonpath-swapping switch block for implementing APUF, alleviating the system delay bias of the two delay chains [20], [21]. A single lookup table (LUT) of FPGA is employed for PDL implementation. By changing the signal input to the LUT, the signal propagation path of the LUT can be adjusted to affect the signal propagation delay of the PDL. As shown in Fig. 5, the *Original Challenge* is the *n-bit* C_1(i.e., $c_{11}, c_{12} \ldots, c_{1(n-1)}, c_{1n}$) mentioned in Fig. 2, and the *Obfuscated Challenge* is the *n-bit* C_O(i.e., $c_{O1}, c_{O2} \ldots, c_{O(n-1)}, c_{On}$) mentioned in Fig. 2. The LFSR-APUF circuit contains n new nonpath-swapping switch units and k tuning units. The upper and lower PDL pairs form a PDL block, which can be divided into two types: 1) switching block and 2) tuning block. The two PDLs of the switching block are applied to the same c_{Oi} ($i = 1, 2, \ldots, n$–1, n), while the two PDLs of the tuning block are applied to the tuning bits t_i^u and t_i^d ($i = 1, 2, \ldots, k$) as independent signals controlled by the user. The challenge controls the switching block, while the user directly controls the tuning block by incorporating an appropriate additional delay to the top or bottom delay chain to counteract the delay deviations caused by asymmetric routing. If the measurement result shows that there are more "1" in the APUF instance responses, it indicates that the top delay chain is too short, then we need to appropriately extend the top delay path by changing the corresponding tuning bit; otherwise, the bottom delay path needs to be appropriately extended if there are more "0." The users generally adjust the tuning bits so that the measured uniformity approaches 50%, while the tuning bits can be fixed. After the tuning bit is fixed, attackers cannot change it.

Since APUF and obfuscation logic are relatively independent in the proposed LFSR-APUF design, it is necessary to *Synthesize* and *Place* and *Route* again after modifying the obfuscation logic by behavioral-level hardware description language (HDL). This leads to unacceptable results influencing the original routing of the APUF circuit. The best way to solve the above problem is to implement an APUF hard macro. The following three steps should be performed to design an unbiased APUF hard macro.

1) Designing a single PDL hard macro.
2) Designing a PDL chain hard macro.
3) Designing an APUF hard macro.

Authorized licensed use limited to: Zhengzhou University. Downloaded on January 02,2024 at 07:24:36 UTC from IEEE Xplore. Restrictions apply.

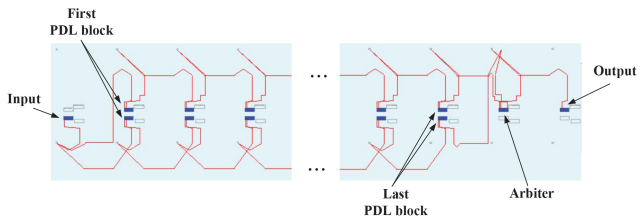

Fig. 6. Placement and routing of the APUF hard macro.

The specific placement and routing of the APUF hard macro are presented in Fig. 6. The routing before the first PDL block and between the last switching block and arbiter (D flip-flop) is asymmetric, while the rest is entirely symmetric. The APUF hard macro can be directly represented as the bottom module to implement the proposed time-variant obfuscation scheme or other schemes without modification.

IV. Experiment Results of the Proposed LFSR-APUF

The proposed LFSR-APUF is implemented on five Xilinx Artix-7 XC7A100T FPGA boards. A computer is used to transmit challenges to the FPGA board and collects responses generated by the PUF. The experimental data is processed by using Python language. In this section, the concept of Hamming distance (HD) is used repeatedly. For the m-bit bitstream X and Y, the HD of X and Y is defined as follows:

$$\mathrm{HD}(X, Y) = \frac{1}{m}\left(\sum_{i=0}^{m-1} X[i] \oplus Y[i]\right) \times 100\%. \tag{8}$$

A. Resistance to Modeling Attack

100000 CRPs are collected for the conventional APUF and the proposed LFSR-APUF to test the modeling attack resistance. Both circuits consist of 64-stage PDL blocks. LR, ES, ANN, and SVM are utilized to evaluate the circuits. 80% of the data set is devoted to training, while the remaining 20% is devoted to testing. Fig. 7 presents the prediction rate of modeling attacks for 64-stage APUF and 64-stage LFSR-APUF under different sizes of training set.

For simplicity, the *Base* value is set to 1. As shown in Fig. 7, the prediction rate of the conventional APUF reaches 85% with fewer than 100 training CRPs. When the training set size is more than 2000, the prediction rate is 91.4%–99.1%. However, since the attacker can only obtain the original external challenge and not the internal obfuscated challenge, the prediction rates of the four attack algorithms for the proposed LFSR-APUF are maintained at 48%–54%. Fig. 8 presents the experimental results of LR attack on APUF and LFSR-APUF with 32 and 64 stages. The prediction rate of LFSR-APUF is much lower than that of APUF for both 32 and 64 stages. Notably, the 32-stage LFSR-APUF has a slightly higher prediction rate than the 64-stage one. Consequently, the obfuscation logic significantly improves the modeling resistance.

LFSR-APUFs with different *Base* values (0, 1, 3, 5, 8, 10, and 20) are implemented and tested. 10 000 CRPs are collected

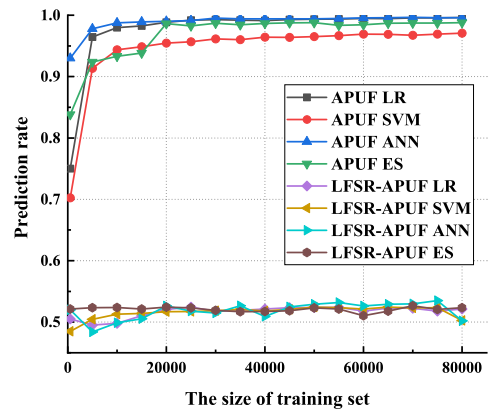

Fig. 7. Result of modeling attacks on APUF and LFSR-APUF.

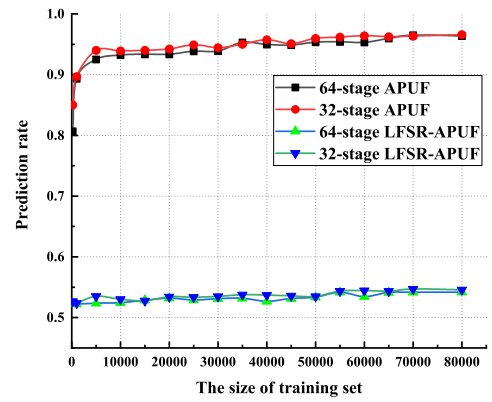

Fig. 8. Result of LR attack on APUF and LFSR-APUF with 32 and 64 stages.

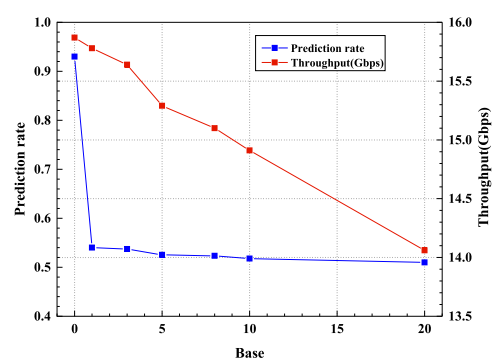

Fig. 9. Prediction rate and throughput rate of 64-stage LFSR-APUF with various *Base* values.

from 64-stage LFSR-APUFs for modeling with LR, and the prediction rates are presented in Fig. 9. When *Base* is 0, the LFSR does not affect the prediction rate, and the prediction rate is approximately 93%; when *Base* is 1, the prediction rate significantly decreases to 54%, demonstrating that the obfuscation mechanism can increase the resistance to modeling attack. When *Base* increases to 10, the prediction rate gradually decreases to nearly 51.79%, almost equivalent to random guess. However, a large *Base* can lead to throughput degradation, as shown by the red line in Fig. 9. Therefore, there is a tradeoff to balance the prediction rate and the throughput. It is recommended to choose the *Base* value as 10 in practical applications.

Authorized licensed use limited to: Zhengzhou University. Downloaded on January 02,2024 at 07:24:36 UTC from IEEE Xplore. Restrictions apply.

TABLE I
COMPARISON OF LFSR-APUF EVALUATION METRICS WITH OTHER WORKS

Property	*Ramdomness*	*Uniqueness*	*Reliability*
Ideal	50 %	50 %	100 %
APUF	53.36 %	48.9 %	91.67%
RPUF [12]	48.9%	52.2 %	94.8%
CP PUF [22]	49.94%	49.98 %	88.9%
LFSR − APUF	51.35 %	49.1 %	99.3%

B. Randomness

Randomness is generally employed to measure the distribution of "0" and "1" in the PUF responses under different input challenges. Randomness is defined as

$$\text{Randomness} = \left(\frac{1}{m}\sum_{i=1}^{m} R_i\right) \times 100\% \tag{9}$$

where m represents the number of PUF response bits, and R_i represents the ith bit response of the PUF instance. Table I shows comparison of LFSR-APUF with other works in various evaluation metrics. If the response bit stream of a PUF is statistically random enough, its randomness should be ideal, i.e., 50%. In our experiment, 80 000 challenges are randomly generated, and the input to the LFSR-APUF is repeated ten times. Finally, the average randomness is approximately obtained as 51.35%, which is close to the desired value.

C. Reliability

Reliability is generally adopted to evaluate the reproducibility of response in different environments of the same challenges, which can be obtained by measuring the intrachip HD of PUF. The Reliability can be expressed as

$$\text{Reliability} = \left(1 - \frac{1}{m}\sum_{i=1}^{m} \frac{\text{HD}(R, R_i)}{n}\right) \times 100\% \tag{10}$$

where m represents the number of tests, n represents the sequence length of PUF response, R represents the reference response value, and R_i represents the value of the ith response under different tests. Ideally, if the same challenges are applied to the same PUF in different environments, the PUF should generate the same response, i.e., the ideal value of reliability is 100%. Reliability is a severe problem for FPGA-based PUF [23], [24], [25]. The original reliability of LFSR-APUF was 94.3%, which was not ideal. A voting module and a soft dark-bit masking module [18] are introduced in the underlying APUF to further improve the reliability. The voting module uses a majority voting strategy to stabilize the output to a response that is more likely to occur. The dark masking module can flag unstable responses and discard them. The unreliable CRPs ultimately discarded accounted for 35.79% of the total CRP set. In our measurement, 80 000 challenges are randomly generated and applied to LFSR-APUF 20 times, respectively, at −20 °C, 0 °C, 20 °C, 25 °C, 30 °C, 50 °C and 70 °C, respectively. As shown in Table I, the average reliability is calculated as 99.3%.

TABLE II
COMPARISON ABOUT HARDWARE OVERHEAD WITH OTHER PUF

Design	*Challenge(bit)*	*Slice LUT*	*Slice Register*
PUF − FSM [11]	128	960	1500
RPUF [12]	128	350	389
RSO − PUF [13]	128	395	176
LFSR − APUF	64	383	399

D. Uniqueness

Uniqueness represents the distinguishability of PUF pairs, and it is a measure of the average number of bit differences among the responses of different PUF instances to the same challenge. The uniqueness of PUF can be obtained by measuring its interchip HD. Uniqueness is defined as follows:

$$\text{Uniqueness} = \left(1 - \frac{2}{k(k-1)}\sum_{i=1}^{k-1}\sum_{j=i+1}^{k} \frac{\text{HD}(R_i, R_j)}{n}\right) \times 100\% \tag{11}$$

where k represents the number of PUF instances tested, R_i and R_j represent the response generated by the ith instance and the jth instance, respectively, under the same test conditions, and n represents the sequence length of all the responses of the PUF instances. Ideally, the uniqueness of PUF should be 50%. In our measurement, 80 000 challenges are randomly generated, and then fed into the LFSR-APUF implemented on five FPGAs. Accordingly, the measured mean uniqueness is obtained as 49.1%.

E. Hardware Overhead

As shown in Fig. 5, the hardware overhead of LFSR-APUF is low as it only contains one LFSR and some necessary control logic circuits more than that in traditional APUF. The LFSR consists of n D-flip flops and several XOR gates, where n indicates the number of challenge bits and the number of XOR gates, which depends on the number of taps. For the LFSR-APUF with the 68-bit challenge (including 64-bit C_1 and 4-bit C_2, as shown in Fig. 2) and 1-bit response, the numbers of Slice LUTs and registers occupied by the 64-stage LFSR-APUF are 383 and 399, respectively. Compared to the baseline APUF, LFSR-APUF adds additional challenge segmentation unit, control unit, vote and mask unit and LFSR unit. Those additional modules consume 18% more hardware resources. A comparison between LFSR-APUF with other MLA-resistant PUFs is presented in Table II. Although the hardware overhead of LFSR-APUF is higher than that of RPUF and RSO-PUF, the MLA-resistant ability of RPUF is lower than that of LFSR-APUF, and RSO-PUF requires an additional consumption of 4 KB of RAM.

V. PROPOSED PUF-BASED AUTHENTICATION PROTOCOL

The proposed protocol consists of a registration phase and an authentication phase. During the registration phase, each IoT device $Device_k$ needs to be registered on the server. In the authentication phase, mutual authentication is required between a device and a server, i.e., each device should prove its authenticity to the server and the server should also prove its legitimacy to the device.

Authorized licensed use limited to: Zhengzhou University. Downloaded on January 02,2024 at 07:24:36 UTC from IEEE Xplore. Restrictions apply.

A. Adversary Model

Since the security of the same authentication protocol is different under different adversary models, we first need to define the adversary capabilities through the adversary model to validate the security of the proposed authentication protocol.

1) The attacker is allowed to eavesdrop, replay, manipulate, and block information traffic on all communication links between the server and the device as well as between the server and the database during the authentication phase.
2) Based on all information traffic among the device, server, and database obtained, the attacker can summarize, deduce and analyze them to find some repetition rules and then perform man-in-the-middle attack and spoofing attack to obtain illegal access.
3) Cloning attacks and intrusive physical attacks are allowed on the devices. Further, the attacker is allowed to conduct MLAs on the device, enabling him/her to make brute force queries on the device through an open interface to predict unknown session information.
4) The server and database are in an insecure environment. An attacker can take full control and attack the server to gain internal information, and he/she also has free access to the information stored in the database.

Before the registration phase, the following assumptions are made: the network environment of the proposed authentication protocol includes mutual authentication between the server and multiple devices. Upon leaving the factory, the server should contain two WPUFs, indicated as WPUF_1 and WPUF_2, which can produce a unique and stable response once powered on. In addition, there are x devices in the authentication network. A unique APUF should be embedded inside each device. For device k, it is defined as APUF_k, and each APUF is unique, random, and stable.

B. Registration Phase

The detailed steps of the registration phase are shown in Fig. 10. These steps are described as follows.

Step 1: WPUF_1 and WPUF_2 generate two server keys after power-on, which are defined as K_{s1} and K_{s2}, respectively.

Step 2: The server assigns a unique identity to each device in the authentication network. The unique identity of $Device_k$ is defined as ID_k. Similarly, the server assigns each device a unique primitive polynomial for the device-specific LFSR design, and the primitive polynomial for $Device_k$ is defined as p_k. The server configures $Cover$ with ID_k to generate the $Cover_k$, which is a unique ultralightweight $Cover$ function for each device.

Step 3: The server encrypts the concatenation of p_k and $Cover_k$ with the encryption key K_{s1}, defined as $Encry_{K_{s1}}(p_k \| Cover_k)$ and abbreviated as Enc_k, in which the symbol $\|$ is used to represent the concatenation of two vectors. Then $Encry_{K_{s1}}(p_k \| Cover_k)$ is stored in the database with ID_k as the index, and some space should be reserved under this index for storing data related to APUF_k. Note that each device in the authentication network is assigned a partition in the database to store relevant information, as shown in Database in Fig. 10.

Step 4: A true random number generator $TRNG$ in the server generates y sets of random numbers $(n_c,\ Base)$. For the round i of authentication $Device_k$, $(n_c,\ Base)$ are defined as $(n_{ci},\ Base_i)$. n_{ci} is n-bit long, equivalent to APUF circuit stage, and y determines the size of the CRP data set used for authentication with the corresponding device. The server removes the n_{ci} whose C_1 is all-zeros or all-ones to prevent the LFSR from getting stuck.

Step 5: For the ith authentication round of $Device_k$, the received $(n_{ci} \| Base_i)$ is input into $LFSR_k$, which determines the seed value and shift count value of the ith authentication round for $LFSR_k$, to generate the corresponding obfuscated challenge n_{Lci}.

Step 6: $LFSR_k$ in $Device_k$ takes the n-bit n_{Lci} generated in the previous step as the seed, named as the initial challenge, to generate n-bit subchallenge, which is defined as $<c_i>_n$, and then $<c_i>_n$ is sent to the underlying APUF_k.

Step 7: APUF_k in $Device_k$ uses the received $<c_i>_n$ to generate an n-bit response, defined as r_i, which is sent to the server. The server checks whether the CRP is reliable.

Step 8: The server encrypts the reliable $(n_{ci} \| Base_i \| r_i)$ with the encryption key K_{s2}, which is defined as $Encry_{K_{s2}}(n_{ci} \| Base_i \| r_i)$, abbreviated as Enc_{ki}, and stores it in the reserved space under the ID_k index in the database.

C. Authentication Phase

After all the legitimate devices are registered on an authentication network, the network enters the authentication phase. Unlike the registration phase which occurs in a secure environment, the entire network is in an insecure environment during the authentication phase. The authentication phase with the ith authentication round of $Device_k$ is shown in Fig. 11. This phase includes the following steps.

Step 1: $Device_k$ sends its own ID_k for the server to initial authentication communication. After receiving the ID_k from $Device_k$, the server simply checks its validity and sends it to the database for requesting the encrypted message related to $Device_k$. By the way, the $Device_k$ configures $Cover$ with ID_k and makes it become a unique $Cover_k$.

Step 2: After receiving the Enc_k about $Device_k$ from the database, the server uses the key K_{s1} generated by WPUF_1 to decrypt it, generating $(p_k \| Cover_k)$, and then the server not only uses the primitive polynomial p_k to configure $LFSR_k$ but also uses $Cover_k$ to configure the private bit conversion function. Note that both $LFSR_k$ and $Cover_k$ are unique and are bound to the $Device_k$.

Step 3: When the server completes the above configuration, $Device_k$ uses its own TRNG to generate (n_d, ind_{i1}), where n_d is a random l-bit for $Cover_k$, and ind_{i1} is used to request the ith round encryption message in the database.

Step 4: After receiving (n_d, ind_{i1}) from the device, the server uses (ID_k, ind_{i1}) to request Enc_{ki} from the database. The database then sends the ith-round encryption message Enc_{ki} about $Device_k$ to the server.

Step 5: After receiving the encrypted message from the database, the server decrypts it with the key K_{s2} generated by WPUF_2, to produce $(n_{ci} \| Base_i \| r_i)$. The server then $Covers$

Authorized licensed use limited to: Zhengzhou University. Downloaded on January 02,2024 at 07:24:36 UTC from IEEE Xplore. Restrictions apply.

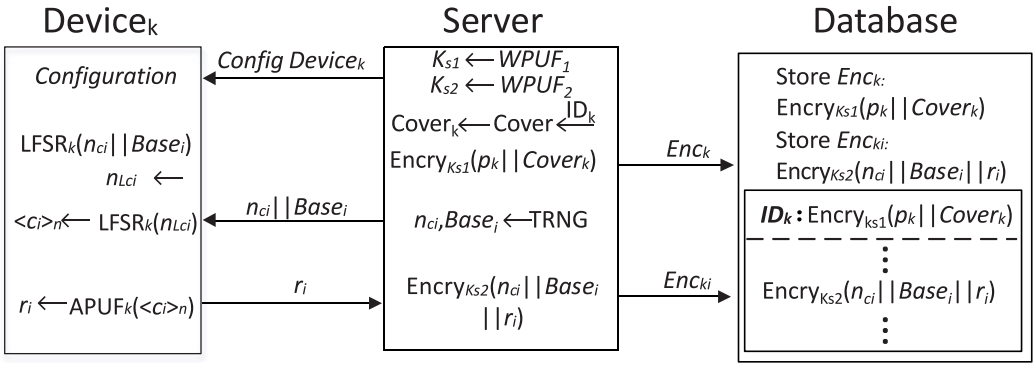

Fig. 10. Registration phase.

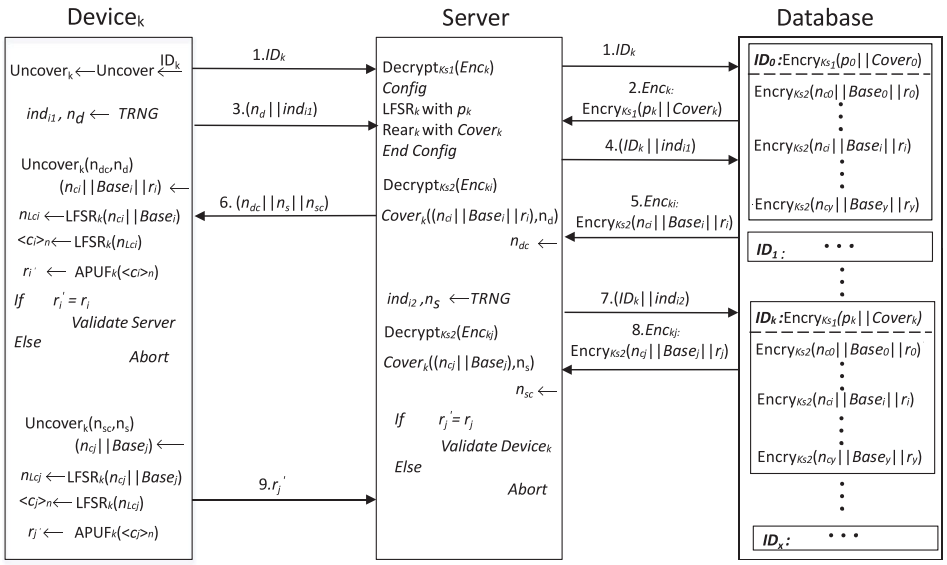

Fig. 11. Authentication phase.

$(n_{ci}\|Base_i\|r_i,\ n_d)$, resulting in n_{dc}. Finally, the server sends the n_{dc} to $Device_k$.

Step 6: The $Device_k$ *Uncovers* the received n_{dc} to obtain the relevant entry, namely, $(n_{ci}\|Base_i\|r_i)$. The $Device_k$ then sends the received $(n_{ci}\|Base_i)$ to $LFSR_k$ for the obfuscated challenge n_{Lci}, and subsequently n_{Lci} is used as the seed of $LFSR_k$ to obtain n n-bit subchallenge $<c_i>_n$. Finally, $<c_i>_n$ is sent to APUF$_k$ to generate an n-bit response r_i. Because the server extracts information about the session during the registration phase, the r_i sent by the legitimate server should be the same as the r_i' generated by $Device_k$. If the r_i and r_i' are the same, the device validates the server.

Step 7: After the server is validated, the server uses its own TRNG to generate (n_s, ind_{i2}) similar to step 3. Then, the server sends the n_s to $Device_k$, and on the other hand, it also sends ind_{i2} to the database to request the related encryption message.

Step 8: After the server receives the Enc_{kj} from the database, it decrypts it with the key K_{s2}, generating $(n_{cj}\|Base_j\|r_j)$. The server then *Covers* the $(n_{cj}\|Base_j)$ with n_s to generate n_{sc}, and sends it to $Device_k$.

Step 9: Once $Device_k$ receives n_s and n_{sc} successively from the server, it *Uncovers* the n_{sc} with n_s to obtain $(n_{cj}\|Base_j)$. Next, $Device_k$ enters $(n_{cj}\|Base_j)$ into LFSR-APUF to generate r_j', which is then sent to the server. After receiving the r_j' sent by $Device_k$, the server compares it with the r_j extracted from the database. If both are equal, the server validates the $Device_k$. In this case, the server and the device have completed mutual authentication.

D. Bit Conversion Through $Cover_k$

The bit conversion function $Cover_k$ has two $l-bit$ arguments. We define l as 128, and define t as 10. The operation process of bit conversion function $Cover_k$ is mainly divided into three stages: 1) bit rearrangement conversion process; 2) parity adjacent cross XOR conversion process; and 3) bit random filling conversion process. In particular, different registered devices have different $Cover_k$ functions, i.e., the bit conversion function $Cover_k$ is unique for $Device_k$.

Next, we define the detailed usage of $Cover_k$ with $X, Y, Z, W, F,$ and O. $X, Y, Z,$ and W, are l-*bit* binary vectors, F is a t-*bit* binary vector, and O is a $(l+t)$-*bit* binary vector, defined as follows:

$$X = x_1, x_2, x_3, \ldots, x_l\,;\ x_i \in \{0, 1\}, i = 1, 2, \ldots, l \tag{12}$$

$$Y = y_1, y_2, y_3, \ldots, y_l\,;\ y_i \in \{0, 1\}, i = 1, 2, \ldots, l \tag{13}$$

$$Z = z_1, z_2, z_3, \ldots, z_l\,;\ z_i \in \{0, 1\}, i = 1, 2, \ldots, l \tag{14}$$

Authorized licensed use limited to: Zhengzhou University. Downloaded on January 02,2024 at 07:24:36 UTC from IEEE Xplore. Restrictions apply.

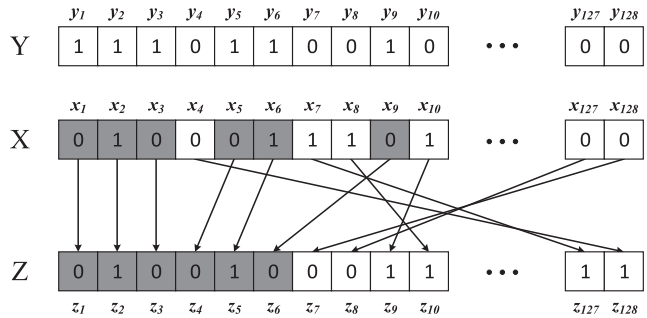

Fig. 12. Bit rearrangement conversion process.

$$W = w_1, w_2, w_3, \ldots, w_l;\ w_i \in \{0, 1\}, i = 1, 2, \ldots, l \quad (15)$$

$$F = f_1, f_2, f_3, \ldots, f_t;\ f_i \in \{0, 1\}, i = 1, 2, \ldots, t \quad (16)$$

$$O = o_1, o_2, o_3, \ldots, o_{(l+t)};\ o_i \in \{0, 1\}, i = 1, 2, \ldots, (l + t). \quad (17)$$

1) Bit Rearrangement: The bit rearrangement conversion process uses $Per(X, Y)$ function [26], and the conversion steps are as follows.

Step 1: Initialize p_x to vector X and p_y to vector Y.

Step 2: Pointers p_x and p_y move bit-by-bit simultaneously from the most significant bit (MSB) to the least significant bit (LSB). During pointer movement, if the bit p_y points to 1, then the bit pointed by p_x is copied into vector Z.

Step 3: Since the vectors X and Y are both l bits, the pointers p_x and p_y reach the LSB at the same time. Similarly, pointers p_x and p_y are returned bit-by-bit from LSB to MSB at the same time, and if p_y refers to bit 0, the bit referred by p_x is copied into vector Z. Note that the bits from X are copied to Z in ascending order.

Step 4: The bit rearrangement conversion process ends when the pointers p_x and p_y return to MSB again.

The specific conversion process is shown in Fig. 12. When p_x and p_y move from MSB to LSB at the same time, if the pointer p_y refers to bit 1, the corresponding bit referred by the pointer p_x in X is represented by a *gray* background. After p_x and p_y return from LSB to MSB at the same time, if the bit referred to by pointer p_y is 0, the corresponding bit referred by pointer p_x in X is represented on a *white* background.

2) Cross XOR: The parity adjacent cross XOR conversion process is as follows.

Step 1: The result of bit rearrangement conversion process, namely, vector Z ($Z \leftarrow Per(X, Y)$), needs to be extracted.

Step 2: The adjacent odd and even bits in vector Z and vector Y are interchanged in pairs according to cross rule.

Step 3: Every two adjacent bits in vector Y are divided into a group, and then l-bit Y can be divided into $l/2$ groups. Next, two bits in every group need to be swapped. The swapped Y is defined as l-bit vector Y'.

Step 4: The vector Y' and vector Z are bitwise XOR converted to obtain the l-bit vector W, as shown in Fig. 13.

In summary, in (18), if i is odd, then $w_i = y_i \oplus z_{i+1}$, otherwise $w_i = y_i \oplus z_{i-1}$, i.e.,

$$W = y_1 \oplus z_2 \| y_2 \oplus z_1 \| y_3 \oplus z_4 \| y_4 \oplus z_3 \ldots y_{127} \oplus z_{128} \| y_{128} \oplus z_{127}. \quad (18)$$

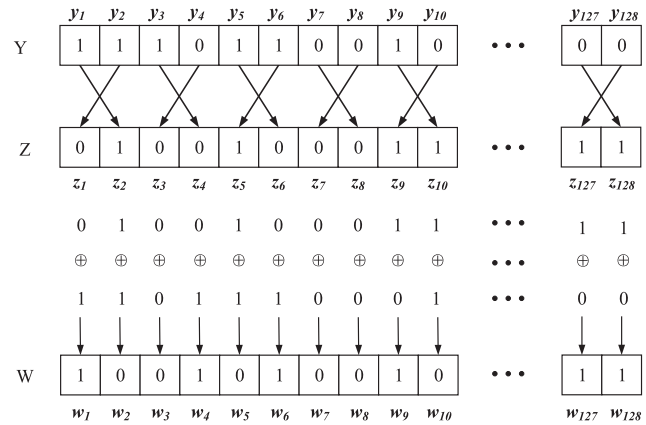

Fig. 13. Parity adjacent cross XOR conversion process.

3) Bit Filling: The $Cover_k$ function defined in the previous section is used for bit filling. The $Cover_k$ function is uniquely bound to Devicek, i.e., the *Cover* functions are different for different devices. The uniqueness of the *Cover* function is mainly reflected in the bit filling conversion process. First t-bit random vector F is generated, which is then randomly filled into l-bit vector W. Obviously, there are $\binom{(l+t)}{t}$ bit-filling schemes. An example of the bit-filling scheme is shown in Fig. 14. According to the definition of l and t above, there are $\binom{(128+10)}{10} \approx 4.9 \times 10^{14}$ bit-filling schemes in total. Such a large number of bit-filling schemes not only ensures that every registered device is assigned a unique *Cover* function but also makes it nearly impossible for an attacker to guess the $Cover_k$ of the embedded $Device_k$. In addition, in every bit-filling scheme, there are 2^t possibilities in the confused filling bits to deceive the attacker, which further enhances the security of the $Cover_k$ function.

VI. Security Analysis and Comparison of the Proposed Authentication Protocol

In this section, we will analyze the different attack methods by which an attacker can compromise the proposed authentication protocol to demonstrate the security properties.

A. Spoofing Attacks

It is assumed that an attacker has sufficient time and storage resources to monitor and store multiple authentication rounds between a server and several devices. Authentication rounds refer to the rounds in which all the legitimate devices and servers have completed mutual authentication after registration, including multiple authentication rounds between the same legitimate device and server, excluding the round in which legitimate devices have not completed mutual authentication with the server. After the above monitoring and storage process is completed, the attacker can impersonate the server, defined as S_F, and start a new round of authentication, namely, the $(i + 1)$th round, with some legitimate device. In addition, the attacker can impersonate any device, defined as D_{kF}. For the sake of explanation, consider that the attacker has already monitored and stored the $(i+1)$th authentication round of $Device_k$.

Authorized licensed use limited to: Zhengzhou University. Downloaded on January 02,2024 at 07:24:36 UTC from IEEE Xplore. Restrictions apply.

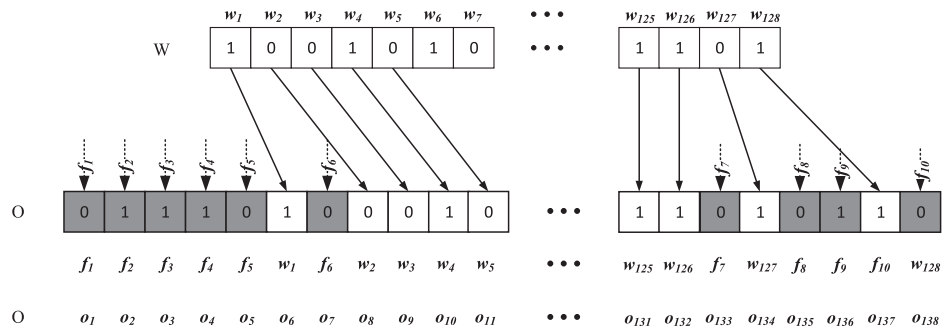

Fig. 14. Bit filling conversion process.

The attacker impersonates the server to launch spoofing attacks as follows.

1) After receiving the ID_k, n_d and $ind_{(i+1)1}$ from $Device_k$, S_F starts to browse the stored $pre-i$ authentication round messages in its own database, to search for if there are exactly the same messages as n_d and $ind_{(i+1)1}$ This is almost impossible. On the one hand, both n_d and $ind_{(i+1)1}$ are generated by the true random number generator without any repetition rule. On the other hand, for length$(n_d) = 138$, length$(ind_{(i+1)1}) = 138$, the probability of each random number occurring is negligible $1/2^{138} \approx 10^{-41}$. Furthermore, the probability of two random numbers reappearing at the same time is even smaller.
2) Because S_F cannot obtain the encryption keys K_{s1} and K_{s2}, it cannot decrypt the encrypted messages from the database. Even if Enc_k and $Enc_{k(i+1)}$ of the $(i+1)$th authentication round from $Device_k$ can be precisely obtained, S_F cannot take any action. S_F can only select a message from the previous authentication rounds and send it to $Device_k$. If S_F sends the ith-round message n_{dc} to $Device_k$, $Device_k$ uncovers the received n_{dc} with n_d, generating sequence $(n_{cif} \| Base_{\text{if}} \| r_{\text{if}})$. $Device_k$ then enters $(n_{cif} \| Base_{\text{if}})$ into LFSR-APUF$_k$ to generate r'_i. Because r_{if} cannot be the same as r'_i, legitimate $Device_k$ rejects the fake server S_F.
3) S_F can also launch a spoofing attack on $Device_k$ with authentication round messages stored in the database from other devices, such as $Device_{k-1}$ and $Device_{k+1}$. This is obviously nearly impossible. If S_F uses authentication round messages from other devices to spoof $Device_k$, the uniqueness and unclonability of the APUF embedded in the device must be considered. Considering that the uniqueness of 64-stage LFSR-APUF measured in the experiment is 49.1%, for LFSR-APUF$_{(k-1)}$, LFSR-APUF$_k$ and LFSR-APUF$_{(k+1)}$, even if the same challenge is applied, the probability of the same responses generated by them is negligible.
4) The attacker impersonates $Device_k$ as D_{k_F} to launch a spoofing attack. D_{k_F} initiates the $(i+1)$th authentication round in the legitimate server. After D_{k_F} validates the legitimate server fraudulently, the server sends n_s and n_{sc} to D_{k_F} to determine if D_{k_F} is trustworthy. Similar to the above case in which legitimate $Device_k$ rejects the fake server S_F, the legitimate server rejects the fake device D_{k_F} because D_{k_F} cannot generate the correct r'_j.

B. Physical Attacks

In the proposed authentication protocol, the server is in such an insecure environment that an attacker can completely control the server. Therefore, an attacker can attack all the devices as well as all the servers in the authentication network.

1) The attacker attempts to launch intrusive probing attacks on legitimate servers to obtain internal configuration information for breaking the proposed authentication protocol. However, probing attack on the servers is meaningless. First, the server does not store any information related to the registered device, including the bit conversion function $Cover_k$, uniquely bound to the device $Device_k$, and the LFSR-APUF$_k$ uniquely embedded in $Device_k$. Second, even if the server is broken during the authentication between server and $Device_k$, the attacker can only obtain the configuration information related to $Device_k$ and cannot obtain $Cover$ of other devices, which has no impact on other device nodes in the authentication network. In addition, the attacker can only obtain $Cover_k$ through probing attacks, and information related to LFSR-APUF$_k$ cannot be obtained. Therefore, the attacker cannot impersonate a server to launch spoofing attacks against $Device_k$. Third, a legitimate server does not store any information related to mutual authentication internally, and essentially relies on encrypted information stored in databases, such as Enc_k and Enc_{ki}. The server uses the keys K_{s1} and K_{s2}, generated by WPUF$_1$ and WPUF$_2$, to decrypt the above encrypted message. The attacker can control the server but cannot get K_{s1} and K_{s2} because WPUF$_1$ and WPUF$_2$ are embedded in the server at the time of manufacturing, and any intrusive probing and tampering with the server can change the key values generated by K_{s1} and K_{s2} [27], making it difficult for the server to properly decrypt the encrypted information from the database.

2) The attacker attempts to launch an intrusive probing attack against legitimate $Device_k$. However, such a probing attack cannot succeed. First, an attacker can obtain the $Cover_k$ circuit inside $Device_k$ by the probing attack, but the $Cover_k$ circuit cannot be generalized to other registered devices. Second, for the underlying LFSR-APUF$_k$ circuit in $Device_k$, the LFSR$_k$ metal wires are attached to the delay path of APUF$_k$. Therefore, once an attacker detects the

Authorized licensed use limited to: Zhengzhou University. Downloaded on January 02,2024 at 07:24:36 UTC from IEEE Xplore. Restrictions apply.

TABLE III
COMPARISON ABOUT PROPERTY WITH OTHER AUTHENTICATION PROTOCOLS-BASED PUF

	$SPUF$ [28]	CSP [29]	$PUF-FSM$ [11]	$PUF-RAKE$ [30]	$MISR$ [31]	RSO [13]	$PUF-IPA$ [32]	$This\ Work$
P1[1]	✓	X	X	X	X	✓	✓	X
P2[2]	X	X	✓	X	X	X	✓	X
P3[3]	X	X	X	X	X	X	X	X
P4[4]	X	X	✓	X	✓	✓	X	X
P5[5]	X	✓	✓	X	X	X	X	X
P6[6]	✓	X	X	X	✓	✓	X	X
P7[7]	✓	X	✓	✓	X	X	✓	✓
P8[8]	✓	✓	✓	✓	✓	✓	✓	✓
P9[9]	✓	X	X	✓	X	X	✓	✓
P10[10]	X	X	X	X	X	X	X	✓

[1] P1: Whether the device contains the NVM module in the authentication protocol.
[2] P2: Whether the device contains the hash logic in the authentication protocol.
[3] P3: Whether the device has the helper data leakage problem in the authentication protocol.
[4] P4: Whether the authentication protocol will be broken assuming the server is controlled by any attacker.
[5] P5: Whether the server contains a secure memory to store the CRPs of PUFs from all devices in the authentication network.
[6] P6: Whether the server contains a secure memory to store the soft models of PUFs from all devices in the authentication network.
[7] P7: Whether the authentication protocol realizes a mutual authentication.
[8] P8: Whether the proposed authentication protocol can resist modeling attacks effectively.
[9] P9: Whether the proposed authentication protocol can resist spoofing attacks including impersonating the server or impersonating the device.
[10] P10: Whether the proposed authentication protocol can resist physical attacks combined with spoofing attacks.

internal structure of LFSR-APUF$_k$ by unencapsulation, the deep submicron structure of APUF$_k$ is destroyed, and the generated responses are inevitably changed.

C. Modeling Attacks

The proposed authentication protocol can resist modeling attacks and brute force attacks against PUF. Brute force attack can be classified as a modeling attack because its primary purpose is to obtain CRP information for modeling.

1) Although the attacker has free access to the database, all CRP information inside the database is stored after encryption with K_{s2}.
2) There is no explicit CRP transmission on the channel during the authentication protocol, and the CRP information about the PUF is basically covered by the proposed *Cover* function.
3) The introduction of LFSR breaks the linear input-output mapping of the standard APUF circuit and greatly enhances the modeling attack resistance of the APUF. In addition, the output of LFSR-APUF is not only related to the challenge $<c>_n$ but also to the *Base*.

Specifically, *Base* value controls the shifting method of LFSR, i.e., different *Base* values correspond to different challenge obfuscation schemes, which changes the original static input-output mapping relationship of LFSR-APUF to a dynamic input-output mapping relationship.

Fourth, the attacker cannot model the PUF accurately to predict the unknown CRPs based on the CRPs obtained by LFSR-APUF. For 64-stage LFSR-APUF with *Base* value of 1, the prediction accuracy of LR, SVM, and ES is only slightly above 50% (see Section IV-A and Fig. 9 for details).

D. Comparison With Other PUF-Based Authentication Protocols

The characteristics of the proposed authentication protocol over other protocols are as follows.

1) The device requires no NVM to store the key, which not only makes it suitable for low-cost platforms but also can resist probing attacks. Some devices based on PUF contain NVM to store some parameters used by the authentication protocol [13], [32], which greatly increases the resource overhead. Most importantly, NVM is vulnerable to physical attacks.
2) The device does not contain heavyweight hash logic but instead adopts the ultralightweight private bit conversion function *Cover*, which reduces the area overhead as well as the overall cost of the device significantly.
3) The device does not use the fuzzy extractor as the error correction means, which reduces the hardware overhead and also ensures that the helper data of the device cannot be leaked, so the PUF is not threatened by the reliability-based side channel attack.
4) Even if the server is completely controlled by the attacker, the attacker cannot obtain any data related to the PUF. In most of the previously reported PUF-based authentication protocols, data related to PUF is stored in the server [13], [31], [32], [34], so once the server is controlled by an attacker, he/she can obtain the PUF-related information and break the entire authentication network.
5) The server does not store the CRPs from PUF of all the devices in the authentication network, but it stores the CRPs and other information in the database after encryption. This not only improves the security of authentication protocols but also eliminates the need for large and expensive on-server memory.
6) The server does not need to store the PUF soft model of all the devices in the authentication network, which reduces the server overhead.
7) Mutual authentication is fully realized, i.e., device-to-server authentication and server-to-device authentication are both satisfied. Some authentication protocols only support server-to-device authentication [13], [31], [34], so they are vulnerable to spoofing attacks by fake

Authorized licensed use limited to: Zhengzhou University. Downloaded on January 02,2024 at 07:24:36 UTC from IEEE Xplore. Restrictions apply.

servers, thereby destroying the entire authentication network.

8) The proposed LFSR-APUF is embedded in the device and is deeply combined with the authentication protocol (see *Base* in Section III-B) to realize the dynamic and time-variant obfuscation mechanism, which can effectively resist modeling attacks. Table III summarizes the discussed comparison.

VII. Conclusion

In this study, we proposed a lightweight authentication protocol against modeling attacks based on a novel LFSR-APUF. First, an LFSR-APUF resistant to modeling attacks was established. The shift strategy of LFSR unit changed with the external challenges, realizing a dynamic and time-variant obfuscation by deep combination with the authentication protocol. The proposed LFSR-APUF could effectively prevent modeling attacks with a prediction rate of 51.79%. Moreover, the introduced obfuscation scheme had no considerable impact on the randomness, reliability, and uniqueness of the conventional APUF. Further, the corresponding strategy may be useful for implementing other APUF-based obfuscation schemes. Second, an ultralightweight PUF-based authentication protocol with a novel bit conversion function was proposed. In particular, the bit conversion *Cover* function was private for every device in the authentication network, which further strengthened the security of the overall authentication network. The proposed authentication protocol ensured the security of the entire authentication network on the premise that the server could be completely controlled by any attacker. In addition, this authentication protocol took the proposed LFSR-APUF and private *Cover* function as the encryption core, which facilitated an exclusive and unique encryption for every device in the authentication network. Overall, the proposed authentication protocol is a feasible and promising strategy for the security of IoT devices.

References

[1] A. Zanella, N. Bui, A. Castellani, L. Vangelista, and M. Zorzi, "Internet of Things for smart cities," *IEEE Internet Things J.*, vol. 1, no. 1, pp. 22–32, Feb. 2014.

[2] A. A. Hasib and A. A. M. Mahmudul Haque, "A comparative study of the performance and security issues of AES and RSA cryptography," in *Proc. 3rd Int. Conf. Converg. Hybrid Inf. Technol.*, vol. 2, 2008, pp. 505–510.

[3] S. Guo et al., "Exploiting the incomplete diffusion feature: A Specialized analytical side-channel attack against the AES and its application to Microcontroller implementations," *IEEE Trans. Inf. Forensics Security*, vol. 9, no. 6, pp. 999–1014, Jun. 2014.

[4] F. Meneghello, M. Calore, D. Zucchetto, M. Polese, and A. Zanella, "IoT: Internet of threats? A survey of practical security vulnerabilities in real IoT devices," *IEEE Internet Things J.*, vol. 6, no. 5, pp. 8182–8201, Oct. 2019.

[5] Y. Gao, S. F. Al-Sarawi, and D. Abbott, "Physical unclonable functions," *Nature Electron.*, vol. 3, no. 2, pp. 81–91, 2020.

[6] J. Guajardo, S. S. Kumar, G.-J. Schrijen, and P. Tuyls, "FPGA intrinsic PUFs and their use for IP protection," in *Proc. Int. Workshop Cryptograph. Hardw. Embedded Syst.*, 2007, pp. 63–80.

[7] U. Rührmair, F. Sehnke, J. Sölter, G. Dror, S. Devadas, and J. Schmidhuber, "Modeling attacks on physical Unclonable functions," in *Proc. 17th ACM Conf. Comput. Commun. Security*, New York, NY, USA, 2010, pp. 237–249.

[8] H. Zhuang, X. Xi, N. Sun, and M. Orshansky, "A strong subthreshold current array PUF resilient to machine learning attacks," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 67, no. 1, pp. 135–144, Jan. 2020.

[9] A. Venkatesh, A. B. Venkatasubramaniyan, X. Xi, and A. Sanyal, "0.3 pJ/bit machine learning resistant strong PUF using subthreshold voltage divider array," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 67, no. 8, pp. 1394–1398, Aug. 2020.

[10] A. Vijayakumar and S. Kundu, "A novel modeling attack resistant PUF design based on non-linear voltage transfer characteristics," in *Proc. Design Autom. Test Europe Conf. Exhibit. (DATE)*, 2015, pp. 653–658.

[11] Y. Gao, H. Ma, S. F. Al-Sarawi, D. Abbott, and D. C. Ranasinghe, "PUF-FSM: A controlled strong PUF," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 37, no. 5, pp. 1104–1108, May 2018.

[12] J. Ye, Y. Hu, and X. Li, "RPUF: Physical unclonable function with randomized challenge to resist modeling attack," in *Proc. IEEE Asian Hardw. Oriented Security Trust (AsianHOST)*, 2016, pp. 1–6.

[13] J. Zhang and C. Shen, "Set-based obfuscation for strong PUFs against machine learning attacks," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 68, no. 1, pp. 288–300, Jan. 2021.

[14] S. T. Choden Konigsmark, D. Chen, and M. D. F. Wong, "PolyPUF: Physically secure self-divergence," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 35, no. 7, pp. 1053–1066, Jul. 2016.

[15] J. Delvaux, "Machine-learning attacks on PolyPUFs, OB-PUFs, RPUFs, LHS-PUFs, and PUF–FSMs," *IEEE Trans. Inf. Forensics Security*, vol. 14, no. 8, pp. 2043–2058, Aug. 2019.

[16] G. T. Becker, "On the pitfalls of using Arbiter-PUFs as building blocks," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 34, no. 8, pp. 1295–1307, Aug. 2015.

[17] U. Rührmair et al., "PUF modeling attacks on simulated and silicon data," *IEEE Trans. Inf. Forensics Security*, vol. 8, no. 11, pp. 1876–1891, Nov. 2013.

[18] S. Satpathy et al., "13fJ/bit probing-resilient 250K PUF array with soft darkbit masking for 1.9% bit-error in 22nm tri-gate CMOS," in *Proc. ESSCIRC*, 2014, pp. 239–242.

[19] M. Goresky and A. Klapper, "Fibonacci and galois representations of feedback-with-carry shift registers," *IEEE Trans. Inf. Theory*, vol. 48, no. 11, pp. 2826–2836, Nov. 2002.

[20] M. Majzoobi, F. Koushanfar, and S. Devadas, "FPGA PUF using programmable delay lines," in *Proc. IEEE Int. Workshop Inf. Forensics Security*, 2010, pp. 1–6.

[21] D. P. Sahoo, R. S. Chakraborty, and D. Mukhopadhyay, "Towards ideal arbiter PUF design on Xilinx FPGA: A practitioner's perspective," in *Proc. Euromicro Conf. Digit. Syst. Design*, 2015, pp. 559–562.

[22] C.-C. Lin and M.-S. Chen, "Enhancing reliability and security: A configurable poisoning PUF against modeling attacks," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 41, no. 11, pp. 4301–4312, Nov. 2022.

[23] T. Tanamoto, S. Yasuda, S. Takaya, and S. Fujita, "Physically Unclonable function using an initial waveform of ring oscillators," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 64, no. 7, pp. 827–831, Jul. 2017.

[24] W.-C. Wang, Y. Yona, S. N. Diggavi, and P. Gupta, "Design and analysis of stability-guaranteed PUFs," *IEEE Trans. Inf. Forensics Security*, vol. 13, no. 4, pp. 978–992, Apr. 2018.

[25] Z. He, W. Chen, L. Zhang, G. Chi, Q. Gao, and L. Harn, "A highly reliable arbiter PUF with improved uniqueness in FPGA implementation using bit-self-test," *IEEE Access*, vol. 8, pp. 181751–181762, 2020.

[26] Y. Tian, G. Chen, and J. Li, "A new ultralightweight RFID authentication protocol with permutation," *IEEE Commun. Lett.*, vol. 16, no. 5, pp. 702–705, May 2012.

[27] B. Gassend, M. V. Dijk, D. Clarke, E. Torlak, S. Devadas, and P. Tuyls, "Controlled physical random functions and applications," *ACM Trans. Inf. Syst. Security*, vol. 10, no. 4, pp. 1–22, 2008.

[28] C. Gu, C. H. Chang, W. Liu, S. Yu, and M. Orneill, "A modeling attack resistant deception technique for securing lightweight-PUF-based authentication," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 40, no. 6, pp. 1183–1196, Jun. 2021.

[29] M. Ebrahimabadi, M. Younis, and N. Karimi, "A PUF-based modeling-attack resilient authentication protocol for IoT devices," *IEEE Internet Things J.*, vol. 9, no. 5, pp. 3684–3703, Mar. 2022.

[30] M. A. Qureshi and A. Munir, "PUF-RAKE: A PUF-based robust and lightweight authentication and key establishment protocol," *IEEE Trans. Depend. Secure Comput.*, vol. 19, no. 4, pp. 2457–2475, Jul./Aug. 2022.

[31] S. S. Zalivaka, A. A. Ivaniuk, and C.-H. Chang, "Reliable and modeling attack resistant authentication of arbiter PUF in FPGA implementation with trinary quadruple response," *IEEE Trans. Inf. Forensics Security*, vol. 14, no. 4, pp. 1109–1123, Apr. 2019.

Authorized licensed use limited to: Zhengzhou University. Downloaded on January 02,2024 at 07:24:36 UTC from IEEE Xplore. Restrictions apply.

[32] M. A. Qureshi and A. Munir, "PUF-IPA: A PUF-based identity preserving protocol for Internet of Things authentication," in *Proc. IEEE 17th Annu. Consum. Commun. Netw. Conf. (CCNC)*, 2020, pp. 1–7.

[33] M.-D. Yu, M. Hiller, J. Delvaux, R. Sowell, S. Devadas, and I. Verbauwhede, "A lockdown technique to prevent machine learning on PUFs for lightweight authentication," *IEEE Trans. Multi-Scale Comput. Syst.*, vol. 2, no. 3, pp. 146–159, Jul.-Sep. 2016.

[34] M. Majzoobi, M. Rostami, F. Koushanfar, D. S. Wallach, and S. Devadas, "Slender PUF protocol: A lightweight, robust, and secure authentication by substring matching," in *Proc. IEEE Symp. Security Privacy Workshops*, 2012, pp. 33–44.

Yao Wang (Member, IEEE) was born in Henan, China, in 1983. He received the M.S. degree from Zhengzhou University, Zhengzhou, China, in 2009, and the Ph.D. degree from the University of Electronic Science and Technology of China (UESTC), Chengdu, China, in 2013.

From 2014 to 2017, he was a Lecturer with UESTC, then he joined Zhengzhou University. Since 2018, he has been an Associate Professor with the School of Information Engineering, Zhengzhou University. His research interests include low-power mixed-signal integrated circuits and hardware security for IoT applications.

Xue Mei was born in Henan, China, in 1998. She received the Bachelor of Science degree from Henan Normal University, Xinxiang, China, in 2020. She is currently pursuing the M.S. degree with Zhengzhou University, Zhengzhou, China.

Her current research interests include hardware security and digital integrated circuits.

Zhengtai Chang was born in Henan, China, in 1997. He received the B.S. degree from Henan Agricultural University, Zhengzhou, China, in 2019, and the M.S. degree from Zhengzhou University, Zhengzhou, in 2022, respectively.

He is currently an FPGA Engineer with XJ Group Corporation. His research interests include hardware security and machine learning.

Wenbing Fan received the Ph.D. degree from the East China University of Science and Technology, Shanghai, China, in 2003.

He is currently a Professor with the School of Information Engineering, Zhengzhou University, Zhengzhou, China. His current research interests include mixed signal circuits and embedded systems.

Benqing Guo (Member, IEEE) received the M.Sc. and Ph.D. degrees in electrical engineering from the University of Electronic Science and Technology of China (UESTC), Chengdu, China, in 2005 and 2011, respectively.

He had multiple years of industrial experience in the development of hardware systems and integrated circuits. Since 2014, he conducted research work on integrated circuits with UESTC. Then, he was a Visiting Scholar with the Microelectronics Group, Pavia University, Pavia, Italy, from 2017 to 2018. In 2020, he joined Chengdu University of Information Technology, Chengdu, China, where he is currently a Researcher. His research interests include RF/analog integrated circuits and hardware security techniques.

Dr. Guo served as a Technical Reviewer for over 15 international journals and conferences, including IEEE TRANSACTIONS ON MICROWAVE THEORY AND TECHNIQUES, IEEE TRANSACTIONS ON VERY LARGE SCALE INTEGRATION, and IEEE ISCAS.

Zhi Quan received the B.S. degree in electronics engineering from Hunan University, Changsha, China, in 2001, and the Ph.D. degree in electronics engineering from the University of York, York, U.K., in 2009.

She then joined the Department of Electrical Engineering, Federal University of Juiz de Fora, Juiz de Fora, Brazil, as a Postdoctoral Researcher. She is currently a Professor with the School of Information Engineering, Zhengzhou University, Zhengzhou, China. Her research interests include high-speed digital circuits, FPGA techniques, and power line communication.

Deepak Kumar Jain (Senior Member, IEEE) received the B.E. degree in electronics and instrumentation from Rajiv Gandhi Proudyogiki Vishwavidyalaya, Bhopal, India, in 2010, the M.Tech. degree in electronic and telecommunication from Jaypee University of Engineering and Technology, Raghogarh-Vijaypur, India, in 2012, and the Ph.D. degree in pattern recognition and intelligent system from the Institute of Automation, University of Chinese Academy of Sciences, Beijing, China, in 2018.

He is currently an Associate Professor with Dalian University of Technology, Dalian, China. He is an Adjunct Professor with the Symbiosis Institute of Technology, Symbiosis International University, Pune, India. He has presented several papers at peer-reviewed conferences and has authored or coauthored numerous studies in science-cited journals. His research interests include the Internet of Things, big data, deep learning, machine learning, pattern recognition, and computer vision.

Authorized licensed use limited to: Zhengzhou University. Downloaded on January 02,2024 at 07:24:36 UTC from IEEE Xplore. Restrictions apply.